1990s, indicate consistently good performance of rollout; see the last section of this chapter for a bibliography. The DP textbook [Ber17a] provides some detailed worked-out examples (Chapter 6, Examples 6.4.2, 6.4.5, 6.4.6, and Exercises 6.11, 6.14, 6.15, 6.16). The price for the performance improvement is extra computation that is typically equal to the computation time of the base heuristic times a factor that is a low order polynomial of N. It is generally hard to quantify the amount of performance improvement, but the computational results obtained from the case studies are consistent with the Newton step interpretations that we discussed in Chapter 1.

The books [Ber19a] (Section 2.5.1) and [Ber20a] (Section 3.1) show that the sequential improvement condition is satisfied in the context of MPC, and is the underlying reason for the stability properties of the MPC scheme. On the other hand the base heuristic underlying the classical form of the MPC scheme is not sequentially consistent (see the preceding references).

Generally, the sequential improvement condition may not hold for a given base heuristic. This is not surprising since any heuristic (no matter how inconsistent or silly) is in principle admissible to use as base heuristic. Here is an example:

Example 2.3.3 (Sequential Improvement Violation)

Consider the 2-stage problem shown in Fig. 2.3.5, which involves two states at each of stages 1 and 2, and the controls shown. Suppose that the unique optimal trajectory is $(x_0, u_0^*, x_1^*, u_1^*, x_2^*)$, and that the base heuristic produces this optimal trajectory starting at x_0. The rollout algorithm chooses a control at x_0 as follows: it runs the base heuristic to construct a trajectory starting from x_1^* and $\tilde{x}_1$, with corresponding costs $H_1(x_1^*)$ and $H_1(\tilde{x}_1)$. If

$$g_0(x_0, u_0^*) + H_1(x_1^*) > g_0(x_0, \tilde{u}_0) + H_1(\tilde{x}_1), \tag{2.15}$$

the rollout algorithm rejects the optimal control u_0^* in favor of the alternative control $\tilde{u}_0$. The inequality above will occur if the base heuristic chooses $\bar{u}_1$ at x_1^* (there is nothing to prevent this from happening, since the base heuristic is arbitrary), and moreover the cost $g_1(x_1^*, \bar{u}_1) + g_2(\tilde{x}_2)$, which is equal to $H_1(x_1^*)$ is high enough.

Let us also verify that if the inequality (2.15) holds then the heuristic is not sequentially improving at x_0, i.e., that

$$H_0(x_0) < \min\big\{g_0(x_0, u_0^*) + H_1(x_1^*),\, g_0(x_0, \tilde{u}_0) + H_1(\tilde{x}_1)\big\}.$$

Indeed, this is true because $H_0(x_0)$ is the optimal cost

$$H_0(x_0) = g_0(x_0, u_0^*) + g_1(x_1^*, u_1^*) + g_2(x_2^*),$$

and must be smaller than both

$$g_0(x_0, u_0^*) + H_1(x_1^*),$$

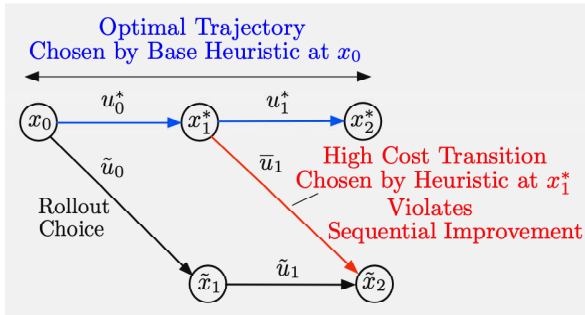

Figure 2.3.5 A 2-stage problem with states $x_1^*, \tilde{x}_1$ at stage 1, and states $x_2^*, \tilde{x}_2$ at stage 2. The controls and corresponding transitions are as shown in the figure. The rollout choice at the initial state x_0 is strictly suboptimal, while the base heuristic choice is optimal. The reason is that the base heuristic is not sequentially improving and makes the suboptimal choice $\overline{u}_1$ at x_1^*, but makes the different (optimal) choice u_1^* when run from x_0.

which is the cost of the trajectory $(x_0, u_0^*, x_1^*, \overline{u}_1, \tilde{x}_2)$, and

$$g_0(x_0, \tilde{u}_0) + H_1(\tilde{x}_1),$$

which is the cost of the trajectory $(x_0, \tilde{u}_0, \tilde{x}_1, \tilde{u}_1, \tilde{x}_2)$.

The preceding example and the monotonicity property (2.14) suggest a simple enhancement to the rollout algorithm, which detects when the sequential improvement condition is violated and takes corrective measures. In this algorithmic variant, called *fortified rollout*, we maintain the best trajectory obtained so far, and keep following that trajectory up to the point where we discover another trajectory that has improved cost (see the next section).

2.3.2 The Fortified Rollout Algorithm

In this section we describe a rollout variant that implicitly enforces the sequential improvement property. This variant, called the *fortified rollout algorithm*, starts at x_0, and generates step-by-step a sequence of states $\{x_0, x_1, \ldots, x_N\}$ and corresponding sequence of controls. Upon reaching state x_k we have the trajectory

$$\overline{P}_k = \{x_0, u_0, \ldots, u_{k-1}, x_k\}$$

that has been constructed by rollout, called *permanent trajectory*, and we also store a *tentative best trajectory*

$$\overline{T}_k = \{x_0, u_0, \ldots, u_{k-1}, x_k, \overline{u}_k, \overline{x}_{k+1}, \overline{u}_{k+1}, \ldots, \overline{u}_{N-1}, \overline{x}_N\}$$

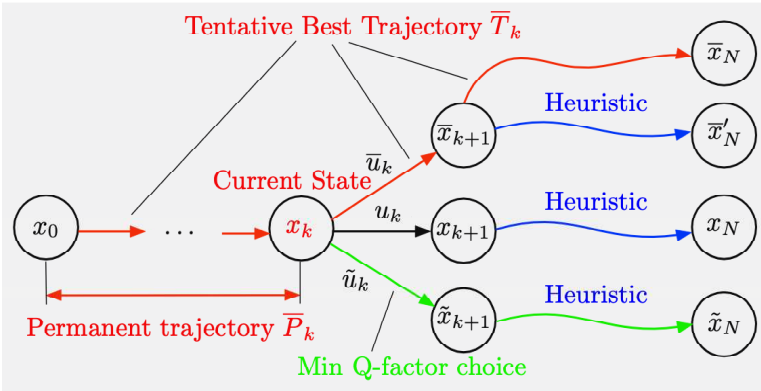

Figure 2.3.6 Schematic illustration of fortified rollout. After k steps, we have constructed the permanent trajectory

$$\overline{P}_k = \{x_0, u_0, \ldots, u_{k-1}, x_k\},$$

and the tentative best trajectory

$$\overline{T}_k = \{x_0, u_0, \ldots, u_{k-1}, x_k, \overline{u}_k, \overline{x}_{k+1}, \overline{u}_{k+1}, \ldots, \overline{u}_{N-1}, \overline{x}_N\},$$

the best end-to-end trajectory computed so far. We now run the rollout algorithm at x_k, i.e., we find the control $\tilde{u}_k$ that minimizes over u_k the sum of $g_k(x_k, u_k)$ plus the heuristic cost from the state $x_{k+1} = f_k(x_k, u_k)$, and the corresponding trajectory

$$\tilde{T}_k = \{x_0, u_0, \ldots, u_{k-1}, x_k, \tilde{u}_k, \tilde{x}_{k+1}, \tilde{u}_{k+1}, \ldots, \tilde{u}_{N-1}, \tilde{x}_N\}.$$

If the cost of the end-to-end trajectory $\tilde{T}_k$ is lower than the cost of $\overline{T}_k$, we add $(\tilde{u}_k, \tilde{x}_{k+1})$ to the permanent trajectory and set the tentative best trajectory to $\overline{T}_{k+1} = \tilde{T}_k$. Otherwise we add $(\overline{u}_k, \overline{x}_{k+1})$ to the permanent trajectory and keep the tentative best trajectory unchanged: $\overline{T}_{k+1} = \overline{T}_k$.

with corresponding cost

$$C(\overline{T}_k) = \sum_{t=0}^{k-1} g_t(x_t, u_t) + g_k(x_k, \overline{u}_k) + \sum_{t=k+1}^{N-1} g_t(\overline{x}_t, \overline{u}_t) + g_N(\overline{x}_N).$$

The tentative best trajectory $\overline{T}_k$ is the end-to-end trajectory that has minimum cost out of all end-to-end trajectories computed up to stage k of the algorithm. Initially, $\overline{T}_0$ is the trajectory generated by the base heuristic starting at the initial state x_0. The idea now is to *discard the suggestion of the rollout algorithm at every state x_k where it produces a trajectory that is inferior to $\overline{T}_k$, and use $\overline{T}_k$ instead* (see Fig. 2.3.6).

In particular, upon reaching state x_k, we run the rollout algorithm as earlier, i.e., for every $u_k \in U_k(x_k)$ and next state $x_{k+1} = f_k(x_k, u_k)$, we

run the base heuristic from x_{k+1}, and find the control $\tilde{u}_k$ that gives the best trajectory, denoted

$$\tilde{T}_k = \{x_0, u_0, \ldots, u_{k-1}, x_k, \tilde{u}_k, \tilde{x}_{k+1}, \tilde{u}_{k+1}, \ldots, \tilde{u}_{N-1}, \tilde{x}_N\}$$

with corresponding cost

$$C(\tilde{T}_k) = \sum_{t=0}^{k-1} g_t(x_t, u_t) + g_k(x_k, \tilde{u}_k) + \sum_{t=k+1}^{N-1} g_t(\tilde{x}_t, \tilde{u}_t) + g_N(\tilde{x}_N).$$

Whereas the ordinary rollout algorithm would choose control $\tilde{u}_k$ and move to $\tilde{x}_{k+1}$, the fortified algorithm compares $C(\overline{T}_k)$ and $C(\tilde{T}_k)$, and depending on which of the two is smaller, chooses $\overline{u}_k$ or $\tilde{u}_k$ and moves to $\overline{x}_{k+1}$ or to $\tilde{x}_{k+1}$, respectively. In particular, if

$$C(\overline{T}_k) \leq C(\tilde{T}_k),$$

the algorithm sets the next state and corresponding tentative best trajectory to

$$x_{k+1} = \overline{x}_{k+1}, \qquad \overline{T}_{k+1} = \overline{T}_k,$$

and if

$$C(\overline{T}_k) > C(\tilde{T}_k),$$

it sets the next state and corresponding tentative best trajectory to

$$x_{k+1} = \tilde{x}_{k+1}, \qquad \overline{T}_{k+1} = \tilde{T}_k.$$

In other words the fortified rollout at x_k follows the current tentative best trajectory $\overline{T}_k$ unless a lower cost trajectory $\tilde{T}_k$ is discovered by running the base heuristic from all possible next states x_{k+1}.† It follows that at every state the tentative best trajectory has no larger cost than the initial tentative best trajectory, which is the one produced by the base heuristic starting from x_0. Moreover, it can be seen that if the base heuristic is sequentially improving, the rollout algorithm and its fortified version coincide. Experimental evidence suggests that it is often important to use the fortified version if the base heuristic is not known to be sequentially improving. Fortunately, the fortified version involves hardly any additional computational cost.

As expected, when the base heuristic generates an optimal trajectory, the fortified rollout algorithm will also generate the same trajectory. This is illustrated by the following example.

† The base heuristic may also be run from a subset of the possible next states x_{k+1}, as in the case where a simplified version of rollout is used; cf. Section 2.3.4. Then fortified rollout will still guarantee a cost improvement property.

Example 2.3.4

Let us consider the application of the fortified rollout algorithm to the problem of Example 2.3.3 and see how it addresses the issue of cost improvement. The fortified rollout algorithm stores as initial tentative best trajectory the optimal trajectory $(x_0, u_0^*, x_1^*, u_1^*, x_2^*)$ generated by the base heuristic at x_0. Then, starting at x_0, it runs the heuristic from x_1^* and $\tilde{x}_1$, and (despite the fact that the ordinary rollout algorithm prefers going to $\tilde{x}_1$ rather than x_1^*) it discards the control $\tilde{u}_0$ in favor of u_0^*, which is dictated by the tentative best trajectory. It then sets the tentative best trajectory to $(x_0, u_0^*, x_1^*, u_1^*, x_2^*)$.

We finally note that the fortified rollout algorithm can be used in a different setting to restore and maintain the cost improvement property. Suppose in particular that the rollout minimization at each step is performed with approximations. For example the control u_k may have multiple independently constrained components, i.e.,

$$u_k = (u_k^1, \ldots, u_k^m), \qquad U_k(x_k) = U_k^1(x_k) \times \cdots \times U_k^m(x_k).$$

Then, to take advantage of distributed computation, it may be attractive to decompose the optimization over u_k in the rollout algorithm,

$$\tilde{\mu}_k(x_k) \in \arg \min_{u_k \in U_k(x_k)} \Big[g_k(x_k, u_k) + H_{k+1}\big(f_k(x_k, u_k)\big) \Big],$$

into an (approximate) parallel optimization over the components u_k^i (or subgroups of these components). However, as a result of approximate optimization over u_k, the cost improvement property may be degraded, even if the sequential improvement assumption holds. In this case by maintaining the tentative best trajectory, starting with the one produced by the base heuristic at the initial condition, we can ensure that the fortified rollout algorithm, even with approximate minimization, will not produce an inferior solution to the one of the base heuristic.

2.3.3 Using Multiple Base Heuristics - Parallel Rollout

In many problems, several promising heuristics may be available. It is then possible to use all of these heuristics in the rollout framework. The idea is to construct a *superheuristic*, which selects the best out of the trajectories produced by the entire collection of heuristics. The superheuristic can then be used as the base heuristic for a rollout algorithm.†

† A related practically interesting possibility is to introduce a partition of the state space into subsets, and a collection of multiple heuristics that are specially tailored to the subsets. We may then select the appropriate heuristic to use on each subset of the partition. In fact one may use a collection of multiple heuristics tailored to each subset of the state space partition, and at each state, select out of all the heuristics that apply, the one that yields minimum cost.

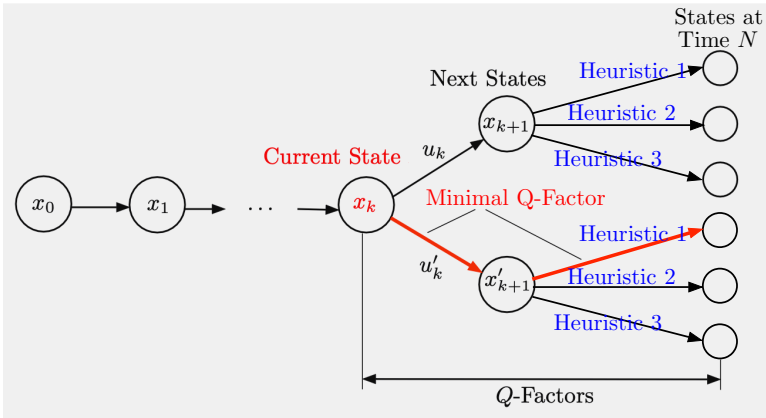

Figure 2.3.7 Schematic illustration of parallel rollout. From every possible next state x_{k+1}, we run each of the heuristics, we compute the Q-factor corresponding to each possible control u_k and heuristic, and select the control with minimal Q-factor. Thus the number of Q-factors that are compared is equal to $m \cdot h$, where m is the number of controls available at state x_k and h is the number of heuristics used in the parallel rollout. In the figure, there are six Q-factors to compare, and the best/minimal Q-factor corresponds to the pair (u'_k, Heuristic 1), as indicated, thus leading to selection of u'_k as the rollout control at x_k.

In particular, let us assume that we have m heuristics, and that the ℓth of these, given a state x_{k+1}, produces a trajectory

$$\tilde{T}^{\ell}_{k+1} = \{x_{k+1}, \tilde{u}^{\ell}_{k+1}, x_{k+2}, \ldots, \tilde{u}^{\ell}_{N-1}, \tilde{x}^{\ell}_{N}\},$$

and corresponding cost $C(\tilde{T}^{\ell}_{k+1})$. The superheuristic then produces at x_{k+1} the trajectory $\tilde{T}^{\ell}_{k+1}$ for which $C(\tilde{T}^{\ell}_{k+1})$ is minimum. The rollout algorithm selects at state x_k the control u_k that minimizes the minimal Q-factor:

$$\tilde{u}_k \in \arg \min_{u_k \in U_k(x_k)} \min_{\ell=1,\ldots,m} \tilde{Q}^{\ell}_k(x_k, u_k),$$

where

$$\tilde{Q}^{\ell}_k(x_k, u_k) = g_k(x_k, u_k) + C(\tilde{T}^{\ell}_{k+1})$$

is the cost of the trajectory $(x_k, u_k, \tilde{T}^{\ell}_{k+1})$. Note that the Q-factors of the different heuristics can be computed independently and in parallel. In view of this fact, the rollout scheme just described is sometimes referred to as *parallel rollout*; see Fig. 2.3.7.

An interesting property, which can be readily verified by using the definitions, is that *if all the heuristics are sequentially improving, the same is true for the superheuristic*, something that is also suggested by the proof of the cost improvement property of Fig. 2.3.4. Indeed, let us write the sequential improvement condition (2.13) for each of the base heuristics

$$\min_{u_k \in U_k(x_k)} \tilde{Q}^{\ell}_k(x_k, u_k) \leq H^{\ell}_k(x_k), \qquad \ell = 1, \ldots, m,$$

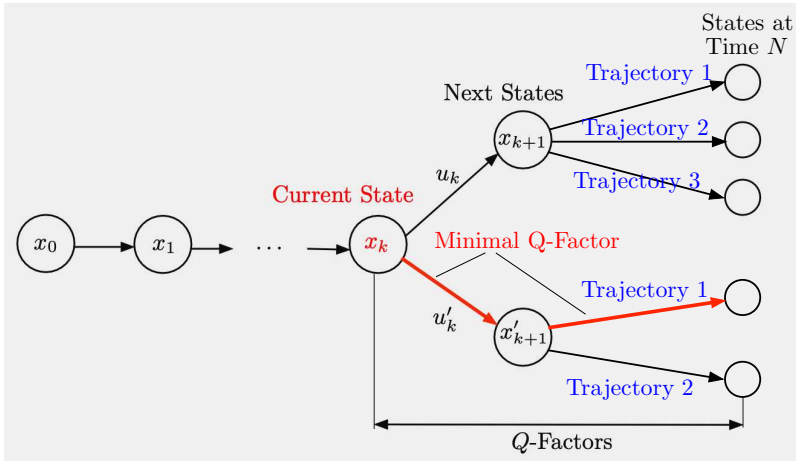

Figure 2.3.8 Schematic illustration of general rollout. From every possible next state x_{k+1}, we run a number of trajectories, we compute the Q-factor corresponding to all the possible (control, trajectory) pairs, and select the control with minimal Q-factor. The number of Q-factors compared is at most $m \cdot h$, where m is the number of controls available at state x_k and h is the maximum number of trajectories generated from each possible next state x_{k+1}. In the figure, there are five Q-factors to compare, and the best/minimal Q-factor corresponds to the pair (u'_k, Trajectory 1), as indicated, thus leading to selection of u'_k as the rollout control at x_k.

where $\tilde{Q}_k^\ell(x_k, u_k)$ and $H_k^\ell(x_k)$ are Q-factors and heuristic costs that correspond to the ℓth heuristic. Then by taking minimum over ℓ, we have

$$\min_{\ell=1,\ldots,m} \min_{u_k \in U_k(x_k)} \tilde{Q}_k^\ell(x_k, u_k) \leq \min_{\ell=1,\ldots,m} H_k^\ell(x_k),$$

for all x_k and k. By interchanging the order of the minimizations of the left side, we then obtain

$$\min_{u_k \in U_k(x_k)} \underbrace{\min_{\ell=1,\ldots,m} \tilde{Q}_k^\ell(x_k, u_k)}_{\text{Superheuristic Q-factor}} \leq \underbrace{\min_{\ell=1,\ldots,m} H_k^\ell(x_k)}_{\text{Superheuristic cost}},$$

which is precisely the sequential improvement condition (2.13) for the superheuristic.

2.3.4 Using Multiple Rollout Trajectories - General Rollout

A generalization of the parallel rollout scheme, called *general rollout*, involves the generation of multiple trajectories starting from each possible next state x_{k+1} (see Fig. 2.3.8). Here, similar to parallel rollout, for each possible x_{k+1}, we compute multiple Q-factors, but we do not require that

each of these Q-factors correspond to a distinct base heuristic. Instead the Q-factors may be associated with multiple trajectories, however generated, which start at x_{k+1} and end at the terminal time N.

Note that some of the trajectories that start from a state x_{k+1} may share some intermediate states from times $k+2$ to N, and they may also be randomly generated. As a result the general rollout framework is well suited for the use of large language models (LLM) as base heuristics. The reason is that the trajectories generated by LLM often involve randomizations and may also involve suboptimization over multiple trajectories. For example, LLM may consider multiple next word choices at selected states, and generate multiple trajectories corresponding to each of these choices.

Similar to parallel rollout, general rollout is not different conceptually from the basic rollout algorithm with a single superheuristic that involves optimization over multiple trajectories starting at any possible next state x_{k+1} (cf. Section 2.3.3). One possibility in general rollout is to carry out this optimization by DP, over a tree of trajectories that is rooted at x_{k+1}. We finally note an important point: with multiple trajectories, the verification of properties such as sequential consistency and sequential improvement, may become complicated. Thus the use of rollout fortification (cf. Section 2.3.2) may be essential to guarantee a cost improvement property.

2.3.5 Simplified Rollout Algorithms

We will now consider a rollout variant, called *simplified rollout*, which is motivated by problems where the control constraint set $U_k(x_k)$ is either infinite or finite but very large. Then the minimization

$$\tilde{\mu}_k(x_k) \in \arg \min_{u_k \in U_k(x_k)} \tilde{Q}_k(x_k, u_k), \tag{2.16}$$

[cf. Eqs. (2.9) and (2.10)], may be unwieldy, since the number of Q-factors

$$\tilde{Q}_k(x_k, u_k) = g_k(x_k, u_k) + H_{k+1}\big(f_k(x_k, u_k)\big)$$

is accordingly infinite or large.

To remedy this situation, we may replace $U_k(x_k)$ with a smaller finite subset $\overline{U}_k(x_k)$:

$$\overline{U}_k(x_k) \subset U_k(x_k).$$

The rollout control $\tilde{\mu}_k(x_k)$ in this variant is one that attains the minimum of $\tilde{Q}_k(x_k, u_k)$ over $u_k \in \overline{U}_k(x_k)$:

$$\tilde{\mu}_k(x_k) \in \arg \min_{u_k \in \overline{U}_k(x_k)} \tilde{Q}_k(x_k, u_k). \tag{2.17}$$

An example is when $\overline{U}_k(x_k)$ results from discretization of an infinite set $U_k(x_k)$. Another possibility is when by using some preliminary approximate optimization, perhaps using a trained neural network or other heuristic method, we can identify a subset $\overline{U}_k(x_k)$ of promising controls, and to

save computation, we restrict attention to this subset. A related possibility is to generate $\overline{U}_k(x_k)$ by some iterative or random search method that explores intelligently the set $U_k(x_k)$ with the aim to minimize $\tilde{Q}_k(x_k, u_k)$ [cf. Eq. (2.16)].

It turns out that the proof of the cost improvement property of Prop. 2.3.2,

$$J_{k,\tilde{\pi}}(x_k) \leq H_k(x_k), \qquad \text{for all } x_k \text{ and } k,$$

goes through if the following modified sequential improvement property holds:

$$\min_{u_k \in \overline{U}_k(x_k)} \tilde{Q}_k(x_k, u_k) \leq H_k(x_k). \tag{2.18}$$

This can be seen by verifying that Eq. (2.18) is sufficient to guarantee that the monotone improvement Eq. (2.14) is satisfied. The condition (2.18) is very simple to satisfy if the base heuristic is sequentially consistent, in which case the control $\overline{u}_k$ selected by the base heuristic satisfies

$$\tilde{Q}_k(x_k, \overline{u}_k) = H_k(x_k).$$

In particular, for the property (2.18) to hold, it is sufficient that $\overline{U}_k(x_k)$ contains the base heuristic choice $\overline{u}_k$.

The idea of replacing the minimization (2.16) by the simpler minimization (2.17) can be extended. In particular, by working through the preceding argument, it can be seen that *any policy*

$$\tilde{\pi} = \{\tilde{\mu}_0, \ldots, \tilde{\mu}_{N-1}\}$$

such that $\tilde{\mu}_k(x_k)$ *satisfies the condition*

$$\tilde{Q}_k\big(x_k, \tilde{\mu}_k(x_k)\big) \leq H_k(x_k),$$

for all x_k *and* k*, guarantees the modified sequential improvement property (2.18), and hence also the cost improvement property.* A prominent example of such an algorithm arises in the multiagent case where u has m components, $u = (u^1, \ldots, u^m)$, and the minimization over $U_k^1(x_k) \times \cdots \times U_k^m(x_k)$ is replaced by a sequence of single component minimizations, one-component-at-a-time; cf. Section 1.6.7. Of course in the multiagent case, the one-component-at-a-time implementation has an additional favorable property: it can be viewed as rollout (without simplification) for a modified but equivalent DP problem (see Section 1.6.7).

2.3.6 Truncated Rollout with Terminal Cost Approximation

An important variation of rollout algorithms is *truncated rollout* with terminal cost approximation. Here the rollout trajectories are obtained by running the base policy from the leaf nodes of the lookahead tree, but they

are truncated after a given number of steps, while a terminal cost approximation is added to the heuristic cost to compensate for the resulting error. This is important for problems with a large number of stages, and it is also essential for infinite horizon problems where the rollout trajectories have infinite length.

One possibility that works well for many problems is to simply set the terminal cost approximation to zero. Alternatively, the terminal cost function approximation may be obtained by using some sophisticated offline training process that may involve an approximation architecture such as a neural network, or by using some heuristic calculation based on a simplified version of the problem. This form of truncated rollout may also be viewed as an intermediate approach between standard rollout where there is no truncation (and hence no cost function approximation), and approximation in value space without any rollout.

2.3.7 Rollout with an Expert - Model-Free Rollout

We will now consider a rollout algorithm for discrete deterministic optimization for the case where *we do not know the cost function and the constraints of the problem*. Instead we have access to a base heuristic, and also a human or software "expert" who can rank any two feasible solutions without assigning numerical values to them.

We consider the general discrete optimization problem of selecting a control sequence $u = (u_0, \ldots, u_{N-1})$ to minimize a function $G(u)$. For simplicity we assume that each component u_k is constrained to lie in a given constraint set U_k, but extensions to more general constraint sets are possible. We assume the following:

(a) A base heuristic with the following property is available: Given any $k < N-1$, and a partial solution $(u_0, \ldots, u_k)$, it generates, for every $\tilde{u}_{k+1} \in U_{k+1}$, a complete feasible solution by concatenating the given partial solution $(u_0, \ldots, u_k)$ with a sequence $(\tilde{u}_{k+1}, \ldots, \tilde{u}_{N-1})$. This complete feasible solution is denoted

$$S_k(u_0, \ldots, u_k, \tilde{u}_{k+1}) = (u_0, \ldots, u_k, \tilde{u}_{k+1}, \ldots, \tilde{u}_{N-1}).$$

The base heuristic is also used to start the algorithm from an artificial empty solution, by generating all components $\tilde{u}_0 \in U_0$ and a complete feasible solution $(\tilde{u}_0, \ldots, \tilde{u}_{N-1})$, starting from each $\tilde{u}_0 \in U_0$.

(b) An "expert" is available that can compare any two feasible solutions u and $\overline{u}$, in the sense that he/she can determine whether

$$G(u) > G(\overline{u}), \qquad \text{or} \qquad G(u) \leq G(\overline{u}).$$

It can be seen that deterministic rollout can be applied to this problem, even though the cost function G is unknown. The reason is that the

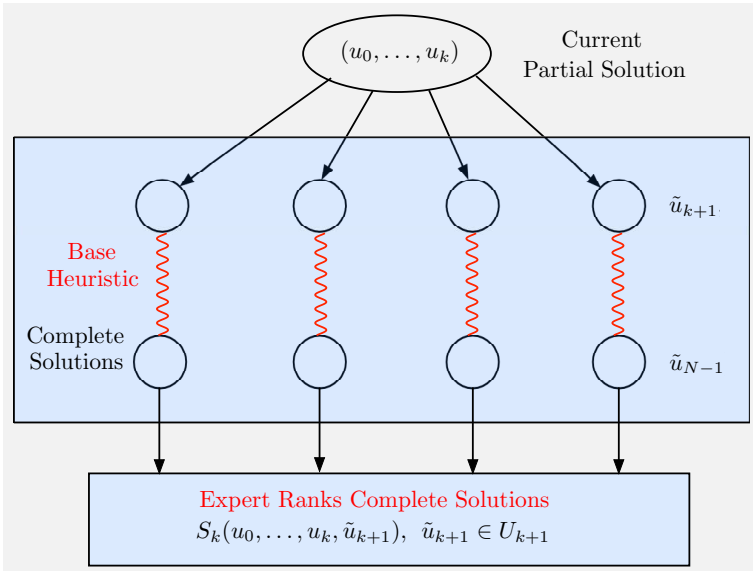

Figure 2.3.9 Schematic illustration of the rollout with an expert for minimizing $G(u)$ subject to

$$u \in U_0 \times \cdots \times U_{N-1}.$$

We assume that we do not know G and/or $U_0, \ldots, U_{N-1}$. Instead we have a base heuristic, which given a partial solution $(u_0, \ldots, u_k)$, outputs all next controls $\tilde{u}_{k+1} \in U_{k+1}$, and generates from each a complete solution

$$S_k(u_0, \ldots, u_k, \tilde{u}_{k+1}) = (u_0, \ldots, u_k, \tilde{u}_{k+1}, \ldots, \tilde{u}_{N-1}).$$

Also, we have a human or software "expert" that can rank any two complete solutions without assigning numerical values to them. The control that is selected from U_{k+1} by the rollout algorithm is the one whose corresponding complete solution is ranked best by the expert.

rollout algorithm uses the cost function only as a means of ranking complete solutions in terms of their cost. Hence, if the ranking of any two solutions can be revealed by the expert, this is all that is needed.† In fact, the constraint sets $U_0, \ldots, U_{N-1}$ need not be known either, as long as they can be generated by the base heuristic. Thus, the rollout algorithm can be described as follows (see Fig. 2.3.9):

We start with an artificial empty solution, and at the typical step, given the partial solution $(u_0, \ldots, u_k)$, $k < N-1$, we use the base heuristic

† Note that for this to be true, it is important that the problem is deterministic, and that the expert ranks solutions using some underlying (though unknown) cost function. In particular, the expert's rankings should have a transitivity property: if u is ranked better than u' and u' is ranked better than u'', then u is ranked better than u''.

to generate all possible one-step-extended solutions

$$(u_0, \ldots, u_k, \tilde{u}_{k+1}), \qquad \tilde{u}_{k+1} \in U_{k+1},$$

and the set of complete solutions

$$S_k(u_0, \ldots, u_k, \tilde{u}_{k+1}), \qquad \tilde{u}_{k+1} \in U_{k+1}.$$

We then use the expert to rank this set of complete solutions. Finally, we select the component u_{k+1} that is ranked best by the expert, extend the partial solution $(u_0, \ldots, u_k)$ by adding u_{k+1}, and repeat with the new partial solution $(u_1, \ldots, u_k, u_{k+1})$.

Except for the (mathematically inconsequential) use of an expert rather than a cost function, the preceding rollout algorithm can be viewed as a special case of the one given earlier. As a result several of the rollout variants that we have discussed so far (rollout with multiple heuristics, simplified rollout, and fortified rollout) can also be easily adapted.

Example 2.3.5 (Active Learning - Dataset Enrichment)

Let us consider a machine learning context whereby we have a dataset that we want to enrich with data selected from another dataset. The enrichment is to be done sequentially, one data point at a time, up to a given maximum size of N data points. We are interested in finding a sequential enrichment policy that optimizes some terminal cost function. This is the active learning context that we discussed briefly in Section 1.7.5.

We can approach the problem in terms of the discrete optimization framework of this section. The state is a partial data set $(u_0, \ldots, u_k)$, consisting of the initial data set, denoted u_0, and the k additional data points $u_1, \ldots, u_k$ that have been selected up to time k. The current state $(u_0, \ldots, u_k)$ is augmented with a data point u_{k+1} from the set of the remaining (unused) data points, and each possible new state is evaluated using a base heuristic and an expert, according to our framework of this section. An interesting variation here is to allow the option for an early termination of the dataset enrichment process; this can be done by allowing a dummy/empty data point selection at each time. An additional possibility for early termination is to introduce a cost for each new data point addition, so that the dataset enrichment process stops naturally, when there is little cost improvement.

Example 2.3.6 (Using a Large Language Model for Rollout)

The problem of minimizing $G(u)$ over a constraint set can be viewed as an N-gram optimization problem, discussed in Example 1.6.2, where the text window consists of a partial solution $(u_0, \ldots, u_k, u_{k+1})$ (preceded by $N-k-2$ "default" words to bring the total to N). We noted in that example that a GPT can be used as a policy that generates next words within the context of N-gram optimization. Thus the problem can be addressed within the model-free rollout framework of this section, whereby a GPT is used as a

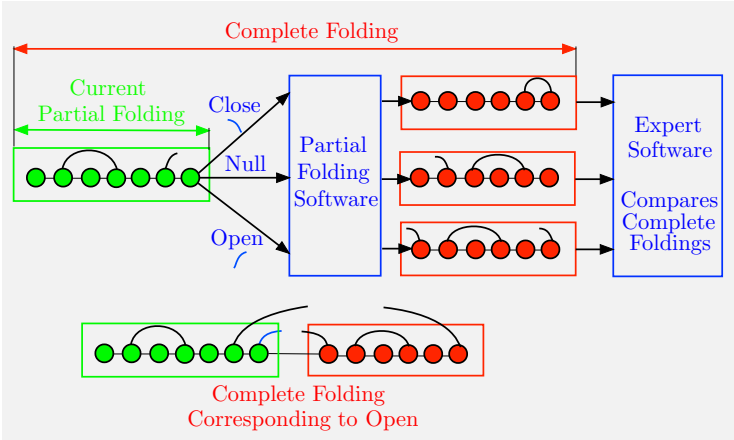

Figure 2.3.10 Schematic illustration of rollout for the RNA folding problem. The current state is the partial folding depicted on the left side. There are at most three choices for control at each state.

base heuristic for completion of partial solutions. The main issues with this approach are how to train a GPT for the problem at hand, and also in the absence of an explicit cost function $G(u)$, how to properly design the expert software for comparing complete solutions. Both of these issues are actively researched at present.

Example 2.3.7 (RNA Folding)

In a classical problem from computational biology, we are given a sequence of nucleotides, represented by circles in Fig. 2.3.10, and we want to "fold" the sequence in an "interesting" way (introduce pairings of nucleotides that result in an "interesting" structure). There are some constraints on which pairings are possible, but we will not go into the details of this (some types of constraints may require the use of the constrained rollout framework of Section 2.5). A common constraint is that the pairings should not "cross," i.e., given a pairing (i_1, i_2) there should be no pairing (i_3, i_4) where either $i_3 < i_1$ and $i_1 < i_4 < i_2$, or $i_1 < i_3 < i_2$ and $i_2 < i_4$. This type of problem has a long history of solution by DP, starting with the paper by Zuker and Stiegler [ZuS81]. There are several formulations, where the aim is to optimize some criterion, e.g., the number of pairings, or the "energy" of the folding. However, biologists do not agree on a suitable criterion, and have developed software to generate "reasonable" foldings, based on semi-heuristic reasoning. We will develop a rollout approach that makes use of such software without discussing their underlying principles.

We formulate the folding problem as a discrete optimization problem involving a pairing decision at each nucleotide in the sequence with at most three choices (open a pairing, close a pairing, do nothing); see Fig. 2.3.10. To apply rollout, we need a base heuristic, which given a partial folding, generates a complete folding (this is the *partial folding software* shown in Fig.